TEMA 10 – GRID

Objetivo del tema

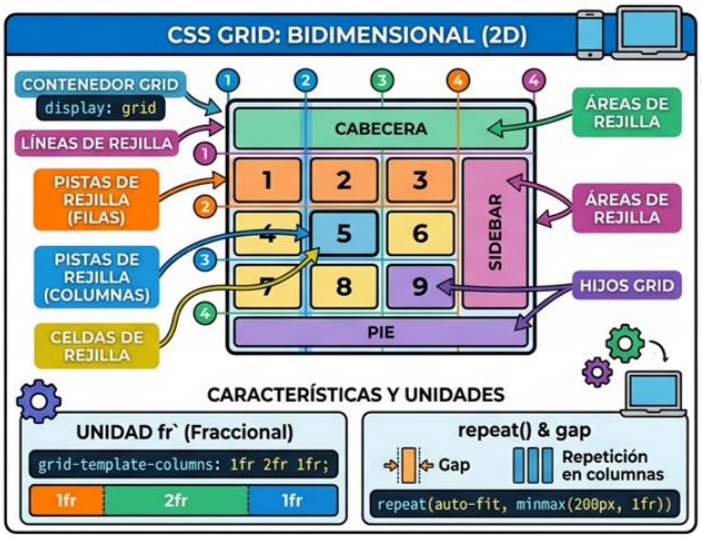
En este tema profundizaremos en las Hojas de Estilo CSS

10.1 GRID

Estos contenidos se encuentran desarrollados en la presentación **UD02 - CSS - Parte10 - Grid**

CSS Grid Layout es el sistema de maquetación bidimensional (*2D*) nativo de CSS. A diferencia de Flexbox, que está pensado para trabajar en un solo eje (fila o columna), Grid permite organizar elementos simultáneamente en filas y columnas.

La idea, por tanto, es similar a la de crear una tabla en Microsoft Word.



Denominamos:

- **Contenedor e Hijos:** Primero, definimos un contenedor mediante display: grid (o inline-grid), que pasa a ser el elemento padre. Todos los elementos hijos directos se convierten en *grid items*.
- **Líneas del Grid (*Grid Lines*):** Las líneas divisorias horizontales y verticales que estructuran la rejilla.
- **Pistas (*Grid Tracks*):** El espacio entre dos líneas consecutivas (es decir, una fila o una columna).
- **Celdas (*Grid Cells*):** La unidad básica de la rejilla (la intersección entre una fila y una columna).
- **Áreas (*Grid Areas*):** Grupo rectangular de celdas adyacentes delimitado por líneas.

Un ejemplo simple:

1. Creamos la estructura HTML del Grid.

```
<div class="grid-container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
    <div>6</div>
</div>
```

2. Añadimos estilos para ordenar el Grid, las celdas, ponerles anchura, etc.

```
.grid-container {
    display: grid;
    grid-template-columns: 200px 1fr 1fr;
    grid-template-rows: 100px 100px;
    gap: 15px;
}
```

En este ejemplo, el marcado como grid-container es el contenedor (padre) mientras que los restantes son los grid-items (hijos).

Para **definir** un elemento como un contenedor, hay que utilizar la regla **display: grid**. Entonces, se le especifican también las propiedades:

- **grid-template-columns**: Cómo debe organizar las columnas, cuántas hay y su tamaño.
- **grid-template-rows**: Cómo debe organizar las filas, cuántas hay y su tamaño.

Podemos hacerlo de varias formas:

1) **Absoluta**: Simplemente le indicamos el número de columnas (o filas) y su tamaño de añadiendo su valor en pixeles.

   Ej: 100 px 100 px → Dos columnas de 100px cada una.

2) **Relativa**: No se especifica el tamaño, se especifica de forma proporcional el tamaño de cada celda en base a las demás y el tamaño total que ocupa esa fila (o columna).

   Ej: 1fr 2fr 1fr → Tres columnas. Dado que 1+2+1 es 4, eso significa que la primera columna ocupa el 25% del especio, la segunda el 50%, y la tercera el 25% restante.

3) **Responsive**: Podemos evitar el código repetitivo mediante funciones.

   Ej: repeat(3, 100px) → Tres filas de 100px

Podemos definir también otras propiedades:

- **grid-auto-columns** y **grid-auto-rows**: Especifican el tamaño predeterminado de columnas y filas cuando no se definen explícitamente en otras propiedades.

- **gap**: Establece el espacio entre las celdas.

- **grid-column** y **grid-row**: Son propiedades atajo (*shorthand*) que se aplican sobre los **elementos hijos** (*grid items*) para definir cuántas columnas o filas deben ocupar dentro de la rejilla.

- **span**: Indica cuántas celdas (filas o columnas) debe abarcar un elemento, sin necesidad de calcular el número exacto de la línea final. De esta forma podemos hacer que un elemento ocupe, por ejemplo, tres columnas. Puede usarse en lugar de grid-column.

```
grid-column: 1 / 4; /* Empieza en la línea 1 y termina en la línea 4 */
grid-column: span 3; /* Ocupa 3 columnas a partir de donde esté colocado */
```

- **justify-content** y **align-content**: propiedades del contenedor padre que controlan la alineación de toda la rejilla en su conjunto dentro del espacio disponible del contenedor. Solo tienen efecto cuando el tamaño total del grid es menor que el tamaño del contenedor que lo envuelve.

- **justify-items** y **align-items**: se aplican en el contenedor padre para controlar cómo se alinea el contenido interno de CADA celda individual.

- **grid-template-areas** permite definir áreas en la cuadrícula del Grid, y asignar nombres a estas áreas. De esta forma, puedes asignar elementos específicos a estas áreas utilizando la propiedad grid-area. Esto facilita la comprensión del diseño y la ubicación de elementos en la cuadrícula. Es particularmente útil en diseños complejos donde la estructura visual es esencial.

  Tienes un **ejemplo** simple de cómo usar grid-area.

10.2 GRID Dinámico

El Grid Dinámico permite que las celdas de un contenedor de cuadrícula se adapten a diferentes tamaños de pantalla de forma **automática**. Obviamente, el objetivo es conseguir diseños responsive.

Se logra combinando tres funciones de CSS Grid: **repeat()**, **auto-fit** (o **auto-fill**) y **minmax()**.

```
.grid-dinamico {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
```

En este caso:

- **minmax(250px, 1fr)**: Define el tamaño mínimo y máximo de cada columna.
    - **Mínimo (250px):** Las columnas nunca serán más estrechas que 250px.
    - **Máximo (1fr):** Si hay espacio sobrante, las columnas se estiran equitativamente para ocuparlo.
- **repeat(..., ...)**: Repite la regla de columnas en lugar de escribirla una por una.
- **auto-fit / auto-fill**: Controla cómo se crean las columnas dinámicas.
    - **Auto-fit**: Estira las tarjetas existentes para que ocupen todo el ancho del contenedor.
    - **Auto-fill**: Mantiene el tamaño mínimo definido y deja el espacio sobrante con celdas invisibles vacías a la derecha.

Ejercicios Grid - Picsum

Picsum (conocido popularmente como **Lorem Picsum**) es un servicio gratuito de imágenes aleatorias (*placeholder images*) diseñado especialmente para desarrolladores web y diseñadores. Funciona de forma análoga al texto *Lorem Ipsum*, pero entregando **fotografías e imágenes reales** listas para usar mediante una simple URL.

Para utilizar Picsum, la referencia a la imagen que quieres usar es una URL similar a esta:

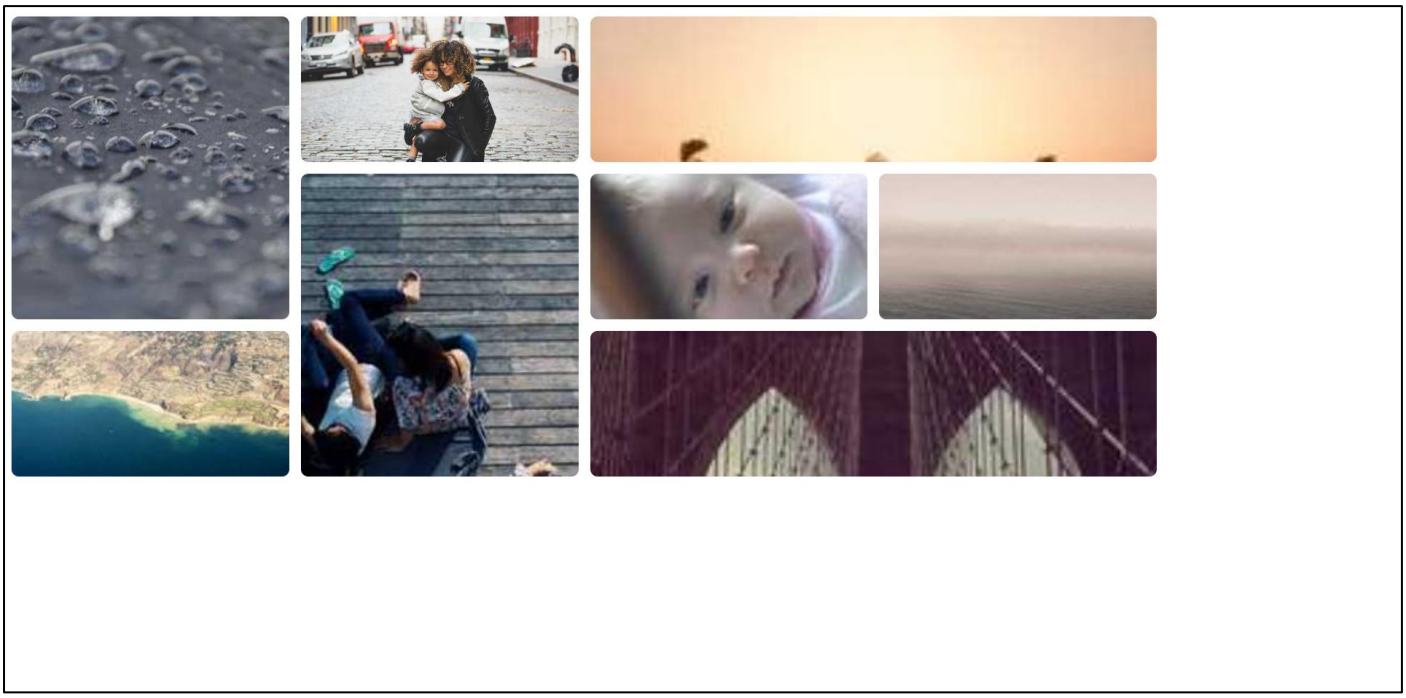
https://picsum.photos/{medidas ancho}/{medidas alto}/?random

Lo que nos generará una imagen de prueba aleatoria del tamaño indicado. Para incrustarla en nuestra web se puede hacer:

```
<img src="https://picsum.photos/400/300" alt="Imagen aleatoria">
```

Recuerda dar medidas distintas para que genere imágenes distintas.

Intenta generar un layout similar a este:



Para el ajuste de las imágenes puedes utilizar la propiedad CSS **object-fit**, que funciona de manera similar a background-position.

Ejercicios Grid – Picsum 2

Realiza el ejercicio anterior, pero utilizando Grid-Areas.

Ejercicios Grid – Dinámico

Construye una página web que muestre diferentes ejemplos de cuadrículas CSS. Cada sección demostrará una propiedad o combinación de propiedades específicas de Grid.

El propósito es experimentar visualmente cómo cambia el comportamiento del diseño según las configuraciones de Grid y comprender cuándo usar cada una.

Tareas a realizar

1. **Crea un archivo index.html y otro style.css.**
2. Añade al menos **4 secciones** en la página, cada una con un título <h2> y varios elementos numerados para distinguirlos.
3. Implementa las siguientes configuraciones:

   a) Grid con auto-fill

   - Usa display: grid.
   - Define columnas con grid-template-columns: repeat(auto-fill, 150px);
   - Observa que el grid genera columnas vacías si no hay suficientes elementos.

4. **b) Grid con auto-fit**

   - Define columnas con grid-template-columns: repeat(auto-fit, 150px);
   - Comprueba que los huecos desaparecen si hay menos elementos de los que caben.

5. **c) Grid dinámico con minmax()**

   - Usa grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
   - Reduce y amplía el tamaño de la ventana para observar cómo las columnas se ajustan automáticamente.

6. **d) Comparativa min-content vs max-content**

   - Crea una cuadrícula con dos columnas:
     grid-template-columns: min-content max-content;
   - Añade dos cajas con textos largos para ver cómo cambian los tamaños de columna.

7. **e) grid-auto-flow: dense**
    - Crea un grid con algunos elementos que ocupen varias filas o columnas (por ejemplo: grid-column: span 2;).
    - Aplica grid-auto-flow: dense; y observa cómo el navegador reacomoda los huecos automáticamente.

Requisitos de formato

- Usa colores o fondos distintos para cada grid, para distinguirlos visualmente.
- Usa gap para separar los elementos.
- Aplica bordes o esquinas redondeadas para mejorar la visibilidad.